Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине «Современные платформы программирования»

Выполнил:
Студент 3 курса
Группы ПО-12
Парфенюк Д. П.
Проверил:
Кулик А. Д.

Брест 2026

Цель работы: закрепить навыки объектно-ориентированного программирования на языке Python

Задание 1:
Реализовать простой класс:

7) Множество символов ограниченной мощности – Предусмотреть возможность объединения двух множеств, вывода на консоль элементов множества, а также метод, определяющий, принадлежит ли указанное значение множеству. Класс должен содержать методы, позволяющие добавлять и удалять элемент в/из множества. Конструктор должен позволять создавать объекты с начальной инициализацией. Мощность множества задается при создании объекта. Реализацию множества осуществить на базе списка. Переопределить метод __eq__, выполняющий сравнение объектов данного типа.

Решение:

```
        """
Модуль для работы с пользовательским классом CharacterSet.
Позволяет создавать ограниченные множества символов и выполнять операции над
ними.
"""

class CharacterSet:
    """
    Класс для представления множества символов с фиксированной максимальной
емкостью.
    """
    def __init__(self, capacity, initial_data=None):
        self.capacity = int(capacity)
        self.items = []
        if initial_data:
            for item in initial_data:
                self.add(item)

    def add(self, char):
        """Добавление элемента в множество"""
        if char not in self.items and len(self.items) < self.capacity:
            self.items.append(char)
            print(f"Успех: Символ '{char}' добавлен.")
        elif len(self.items) >= self.capacity:
            print(f"Ошибка: Лимит ({self.capacity}) исчерпан!")
        else:
            print(f"Инфо: Символ '{char}' уже есть в множестве.")

    def remove(self, char):
        """Удаление элемента из множества"""
        if char in self.items:
            self.items.remove(char)
            print(f"Успех: Символ '{char}' удален.")
        else:
            print(f"Ошибка: Символа '{char}' нет в этом множестве.")

    def belongs(self, char):
        """Принадлежит ли значение множеству (True/False)"""
        return char in self.items

    def union(self, other):
        """Объединение двух множеств"""
        new_set = CharacterSet(self.capacity + other.capacity)
        for item in self.items:
```

```
            new_set.add(item)
        for item in other.items:
            new_set.add(item)
        return new_set

    def __str__(self):
        """Вывод элементов на консоль"""
        if not self.items:
            return "{пусто}"
        return "{" + ", ".join(self.items) + "}"

    def __eq__(self, other):
        """Сравнение объектов типа CharacterSet"""
        if not isinstance(other, CharacterSet):
            return False
        return sorted(self.items) == sorted(other.items)


print("Начальный ввод ")
cap1 = input("Введите лимит для множества 1: ")
set1 = CharacterSet(cap1)

cap2 = input("Введите лимит для множества 2: ")
set2 = CharacterSet(cap2)

while True:
    print("\n" + "=" * 30)
    print(f"множество 1: {set1}")
    print(f"множество 2: {set2}")
    print("-" * 30)
    print("1. Добавить символ в множество 1")
    print("2. Добавить символ в множество 2")
    print("3. Удалить символ из множества 1")
    print("4. Проверить принадлежность (множество 1)")
    print("5. Объединить множества (1 + 2)")
    print("6. Сравнить множества (1 == 2)")
    print("0. Выход")

    choice = input("\nВыберите действие: ")

    if choice == "1":
        user_input = input("Введите символ: ")
        set1.add(user_input)

    elif choice == "2":
        user_input = input("Введите символ: ")
        set2.add(user_input)

    elif choice == "3":
        user_input = input("Какой символ удалить из Множества 1?: ")
        set1.remove(user_input)

    elif choice == "4":
        user_input = input("Какой символ ищем в Множестве 1?: ")
        if set1.belongs(user_input):
            print(f"Результат: Символ '{user input}' входит в Множество 1.")
        else:
            print("Результат: Символа нет.")

    elif choice == "5":
        result_set = set1.union(set2)
        print(f"Результат объединения: {result_set}")
        print("(Новое множество создано на основе суммы лимитов)")
```

```
    elif choice == "6":
        if set1 == set2:
            print("Результат: Множества равны!")
        else:
            print("Результат: Множества разные.")

    elif choice == "0":
        print("Завершение работы.")
        break
    else:
        print("Неверный ввод, попробуйте еще раз.")
```

```
Выберите действие: 5
Успех: Символ 'a' добавлен.
Успех: Символ 'd e e f' добавлен.
Успех: Символ 'a b c d' добавлен.
Результат объединения: {a, d e e f, a b c d}
(Новое множество создано на основе суммы лимитов)

==============================
множество 1: {a, d e e f}
множество 2: {a b c d}
------------------------------
```

Задание 2:

6) Система Телефонная станция. Абонент оплачивает Счет за разговоры и Услуги, может попросить Администратора сменить номер и отказаться от услуг. Администратор изменяет номер, Услуги и временно отключает Абонента за неуплату.

Решение:

```
"""
Модуль для управления системой абонентов телефонной станции.
Включает описание услуг, данные абонентов и функции администратора.
"""
from dataclasses import dataclass


@dataclass
class Service:
    """Класс для представления услуги телефонной связи."""
    name: str
    price: float

    def __str__(self):
        return f"{self.name} ({self.price} руб.)"


class Subscriber:
    """Класс для хранения данных об абоненте и его балансе."""
    def __init__(self, name, phone_number):
```

```
        self.name = name
        self.phone_number = phone_number
        self.balance = 0
        self.services = []
        self.is_active = True

    def pay_bill(self, amount):
        """Абонент оплачивает счет"""
        self.balance += amount
        print(f"[Абонент]: Счет оплачен на {amount} руб. Баланс:
{self.balance}")

    def __str__(self):
        status = "Подключен" if self.is_active else "ОТКЛЮЧЕН ЗА НЕУПЛАТУ"
        srv_names = [srv.name for srv in self.services]
        serv_list = ", ".join(srv_names) if self.services else "нет"

        return (f"\n Данные системы \n"
                f"Абонент: {self.name}\n"
                f"Номер: {self.phone_number}\n"
                f"Баланс: {self.balance} руб.\n"
                f"Статус: {status}\n"
                f"Услуги: {serv_list}\n")


class Administrator:
    """Класс для выполнения административных действий над абонентами."""
    def change_number(self, subscriber, new_number):
        """Администратор изменяет номер"""
        subscriber.phone_number = new_number
        print(f"[Админ]: Номер абонента {subscriber.name} изменен.")

    def add_service(self, subscriber, new_srv):
        """Администратор подключает услугу"""
        # Исправлено: аргумент переименован в new_srv
        subscriber.services.append(new_srv)
        print(f"[Админ]: Услуга '{new_srv.name}' добавлена.")

    def remove_service(self, subscriber, service_name):
        """Администратор отключает услугу по просьбе абонента"""
        for srv in subscriber.services:
            if srv.name.lower() == service_name.lower():
                subscriber.services.remove(srv)
                print(f"[Админ]: Услуга '{service_name}' удалена.")
                return
        print(f"[Админ]: Услуга '{service_name}' не найдена.")

    def manage_connection(self, subscriber):
        """Администратор временно отключает за неуплату"""
        if subscriber.balance < 0:
            subscriber.is_active = False
            print(f"[Админ]: Абонент {subscriber.name} ОТКЛЮЧЕН.")
        else:
            subscriber.is_active = True
            print(f"[Админ]: Абонент {subscriber.name} активен.")


all_services = [
    Service("Интернет", 400),
    Service("СМС-пакет", 100),
    Service("Межгород", 250)
]

print(" Вход в систему Телефонной Станции")
```

```python
u_name = input("Введите имя нового абонента: ")
u_phone = input("Введите номер телефона: ")
sub = Subscriber(u_name, u_phone)
admin = Administrator()

while True:
    print("\nМЕНЮ:")
    print("1. Показать состояние")
    print("2. Оплатить счет")
    print("3. Списать за разговоры")
    print("4. Сменить номер")
    print("5. Подключить услугу")
    print("6. Отказаться от услуги")
    print("7. Проверить статус")
    print("0. Выход")

    choice = input("\nВыберите действие: ")

    if choice == "1":
        print(sub)
    elif choice == "2":
        money = float(input("Введите сумму оплаты: "))
        sub.pay_bill(money)
    elif choice == "3":
        debt = float(input("Введите стоимость разговоров: "))
        sub.balance -= debt
        print(f"Баланс: {sub.balance} руб.")
    elif choice == "4":
        new_num = input("Введите новый номер: ")
        admin.change_number(sub, new_num)
    elif choice == "5":
        print("\nСписок услуг:")
        # Исправлено: переменная цикла переименована в srv_item
        for i, srv_item in enumerate(all_services):
            print(f"{i+1}. {srv_item}")
        s_idx = int(input("Выберите номер услуги: ")) - 1
        admin.add_service(sub, all_services[s_idx])
    elif choice == "6":
        name_to_del = input("Введите название услуги: ")
        admin.remove_service(sub, name_to_del)
    elif choice == "7":
        admin.manage_connection(sub)
    elif choice == "0":
        break
    else:
        print("Неверный ввод.")
```

```
2. Оплатить счет (Абонент)
3. Списать за разговоры (Симуляция долга)
4. Попросить сменить номер (Админ изменит)
5. Подключить услугу (Админ добавит)
6. Отказаться от услуги (Админ удалит)
7. Проверить оплату и статус (Админ отключит/включит)
0. Выход

Выберите действие: 3
Введите стоимость разговоров: 100
Баланс уменьшен на 100.0 руб. Текущий баланс: -100.0
```

```
МЕНЮ:
1. Показать состояние (Счет и Услуги)
2. Оплатить счет (Абонент)
3. Списать за разговоры (Симуляция долга)
4. Попросить сменить номер (Admin изменит)
5. Подключить услугу (Admin добавит)
6. Отказаться от услуги (Admin удалит)
7. Проверить оплату и статус (Admin отключит/включит)
0. Выход

Выберите действие: 7
[Admin]: Абонент Daniil ОТКЛЮЧЕН (отрицательный баланс).
```

Вывод: закрепил навыки объектно-ориентированного программирования на языке Python